



UNIVERSIDAD DE GUAYAQUIL
FACULTAD DE INGENIERIA INDUSTRIAL
CARRERA INGENIERIA TELEMATICA

PROYECTO DE PLANIFICACIÓN Y GESTIÓN DE REDES

Evaluación de Prestaciones de un Servicio Web

Docente: Ing. Ericka Oyague B. MSc.

Participantes:
Lucas Sánchez Isaías Joel
Marmolejo Sancan Alisson Johana
Peñafiel Alvarado Edinson Enrique
Romero Merchan Carolina Yamilex

Período: 2025 - CII
Guayaquil, 2026

Tabla de contenido

Objetivo General	4
Introducción	4
1. Estado del Arte	5
1.1 Arquitecturas Comunes de Servidores Web	5
1.2 Herramientas de Monitorización y Evaluación de Rendimiento	5
1.3 Tipos de Pruebas de Rendimiento.....	6
2. Implementación de la Propuesta	7
2.1 Levantamiento del Servidor Web	7
2.2 Servidor con Soporte para Múltiples Peticiones.....	9
2.3 Integración con Sistema de Colas de Mensajes (RabbitMQ)	10
2.4 Monitorización del Servicio	12
2.5 Simulación de Carga	15
3. Resultados	18
3.1 Gráficas de Monitorización y Análisis	18
3.2 Análisis de Conexión TCP con Wireshark	22
3.3 Pruebas de Estrés y Carga.....	25
4. Conclusiones y Recomendaciones	31
4.1 Conclusiones	31
4.2 Recomendaciones	31

<i>Ilustración 1: Servidor Encendido - TechStore.....</i>	<i>8</i>
<i>Ilustración 2: Terminal realizando peticiones - TechStore - ngrok</i>	<i>8</i>
<i>Ilustración 3: Pequeña Prueba de Carga - Locust.....</i>	<i>9</i>
<i>Ilustración 4: Analisis de cola de mensajes - RabbitMQ.....</i>	<i>11</i>
<i>Ilustración 5: Terminal del consumidor - Graficas RabbitMQ.....</i>	<i>11</i>
<i>Ilustración 6: Grafica de CPU utilization - Windows.....</i>	<i>12</i>
<i>Ilustración 7: Rendimiento de la Memoria RAM - Windows</i>	<i>13</i>
<i>Ilustración 8: Rendimiento de la red - Windows</i>	<i>13</i>
<i>Ilustración 9: Grafica CPU - Grafana</i>	<i>14</i>
<i>Ilustración 10: Grafica Memoria RAM - Grafana</i>	<i>14</i>
<i>Ilustración 11: Interfaz de Locust configurando la simulación de carga.....</i>	<i>15</i>
<i>Ilustración 12: Resultados de Locust mostrando peticiones por segundo y tiempos de respuesta</i>	<i>16</i>
<i>Ilustración 13: Resultados de Locust mostrando peticiones por segundo y tiempos de respuesta</i>	<i>17</i>
<i>Ilustración 14: Monitorizacion de CPU durante pruebas de rendimiento - Zabbix</i>	<i>18</i>
<i>Ilustración 15: Monitorizacion de Memoria RAM durante las pruebas - Zabbix</i>	<i>19</i>
<i>Ilustración 16: Gráfica de número de peticiones atendidas por ruta</i>	<i>19</i>
<i>Ilustración 17: Pruebas de estrés con exceso de solicitudes - Locust.....</i>	<i>22</i>
<i>Ilustración 18: Captura de Wireshark mostrando el Three-Way Handshake TCP</i>	<i>23</i>
<i>Ilustración 19: Carga con 10 usuarios - Locust.....</i>	<i>26</i>
<i>Ilustración 20: Grafica durante la carga de 10 usuarios - Locust</i>	<i>26</i>
<i>Ilustración 21: Carga de 50 usuarios (spawn rate de 5) – Locust</i>	<i>27</i>
<i>Ilustración 22: Grafica de la carga de 50 usuarios - Locust</i>	<i>27</i>
<i>Ilustración 23: Carga con 100 usuarios – Locust.....</i>	<i>27</i>
<i>Ilustración 24: Grafica de prueba con 100 usuarios – Locust</i>	<i>28</i>
<i>Ilustración 25: Carga de prueba con 200 usuarios - Locust.....</i>	<i>29</i>
<i>Ilustración 26: Grafica de peticiones con 200 usuarios - Locust.....</i>	<i>30</i>
<i>Ilustración 27: Tabla de valores en el terminal - Locust</i>	<i>30</i>

Objetivo General

Evaluar las prestaciones de un servicio web levantado por el estudiante, utilizando herramientas de análisis de rendimiento y monitorización de red.

Introducción

En este proyecto se implementó un servicio web funcional y escalable denominado TechStore, una tienda online de productos tecnológicos desarrollada con Python y el framework Flask. El propósito principal es analizar el comportamiento del servicio frente a múltiples peticiones simultáneas, evaluando su rendimiento bajo condiciones controladas.

La arquitectura del sistema considera el flujo de peticiones entre clientes y servidores, incorporando herramientas de monitorización que permiten evaluar el rendimiento del servicio. Adicionalmente, se integró un sistema de colas de mensajes mediante RabbitMQ para desacoplar la producción y consumo de peticiones, y se realizaron pruebas de carga para identificar posibles cuellos de botella.

1. Estado del Arte

1.1 Arquitecturas Comunes de Servidores Web

Modelo Cliente-Servidor

El modelo cliente-servidor es la arquitectura base de toda comunicación web. En este modelo, el cliente, que puede ser un navegador web, una aplicación móvil o un script automatizado, realiza peticiones al servidor mediante el protocolo HTTP o HTTPS. El servidor recibe, procesa y devuelve una respuesta apropiada al cliente.

Este modelo define claramente dos roles diferenciados: el cliente, que inicia la comunicación solicitando recursos o servicios, y el servidor, que permanece en espera de peticiones para procesarlas y responderlas. La comunicación se establece mediante mensajes estructurados que incluyen métodos HTTP como GET, POST, PUT y DELETE, códigos de estado de respuesta y el contenido transferido.

Escalado Vertical vs. Escalado Horizontal

Característica	Escalado Vertical	Escalado Horizontal
Definición	Aumentar recursos del mismo servidor (CPU, RAM)	Agregar más servidores en paralelo
Implementación	Upgrade de hardware	Múltiples instancias del servicio
Costo	Alto costo inicial	Más flexible y económico
Límite	Límite físico del hardware	Prácticamente ilimitado
Uso típico	Bases de datos pequeñas	Aplicaciones web de alta demanda

1.2 Herramientas de Monitorización y Evaluación de Rendimiento

Las herramientas de monitorización permiten capturar y analizar en tiempo real el comportamiento de los sistemas, proporcionando métricas esenciales como uso de CPU, consumo de memoria, tiempos de respuesta y número de peticiones atendidas.

Herramienta	Descripción	Uso principal
Zabbix	Plataforma empresarial de monitorización de red y sistemas	Monitorización de infraestructura
Grafana	Herramienta de visualización y análisis de métricas	Dashboards y gráficas en tiempo real
Prometheus	Sistema de recolección de métricas con modelo pull	Métricas de aplicaciones
Netdata	Monitorización en tiempo real de alto rendimiento	Análisis de rendimiento del sistema
Nagios	Sistema de alertas y monitorización de servicios	Alertas y notificaciones

1.3 Tipos de Pruebas de Rendimiento

Tipo de Prueba	Descripción	Objetivo
Prueba de Carga	Simula un número de usuarios esperado durante un tiempo determinado	Verificar comportamiento bajo carga normal
Prueba de Estrés	Incrementa la carga hasta encontrar el punto de quiebre del sistema	Identificar el límite máximo del servidor
Prueba de Escalabilidad	Evalúa cómo responde el sistema al aumentar progresivamente la carga	Planificar capacidad futura
Prueba de Pico (Spike)	Genera un aumento repentino y masivo de tráfico	Evaluar recuperación ante tráfico inesperado

2. Implementación de la Propuesta

2.1 Levantamiento del Servidor Web

Se implementó un servidor web funcional utilizando Python con el framework Flask. El servicio simula una tienda online denominada TechStore, que ofrece productos tecnológicos y permite realizar operaciones de consulta, búsqueda, gestión de carrito y procesamiento de órdenes.

Tecnologías utilizadas

- Lenguaje de programación: Python 3.8.10
- Framework web: Flask 3.0.3
- Servidor de producción: Waitress
- Sistema operativo: Windows 11

Rutas implementadas en el servidor

Ruta	Método	Descripción	Tiempo simulado
GET /	GET	Página principal de la tienda (HTML)	Inmediato
GET /productos	GET	Lista todos los productos disponibles	100 ms
GET /productos/<id>	GET	Detalle de un producto específico	50 ms
GET /buscar?q=	GET	Búsqueda de productos por nombre	200 ms
GET /carrito	GET	Visualización del carrito actual	Inmediato
GET /carrito/agregar/<id>	GET	Agregar producto al carrito	80 ms
GET /orden	GET	Procesar orden de compra	500 ms
GET /metricas	GET	Métricas del servidor en tiempo real	Inmediato

Evidencia del servidor en funcionamiento

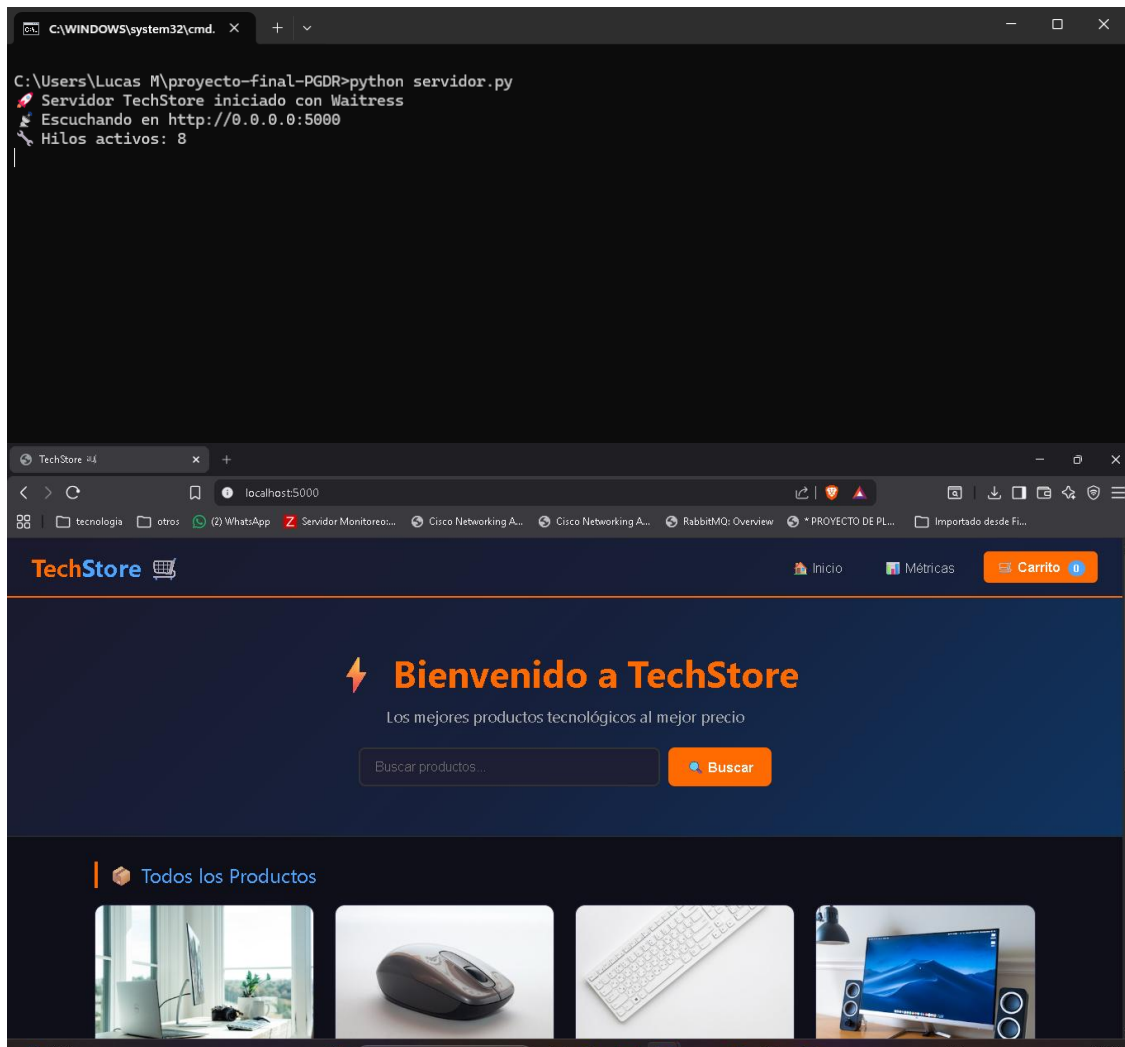


Ilustración 1: Servidor Encendido - TechStore

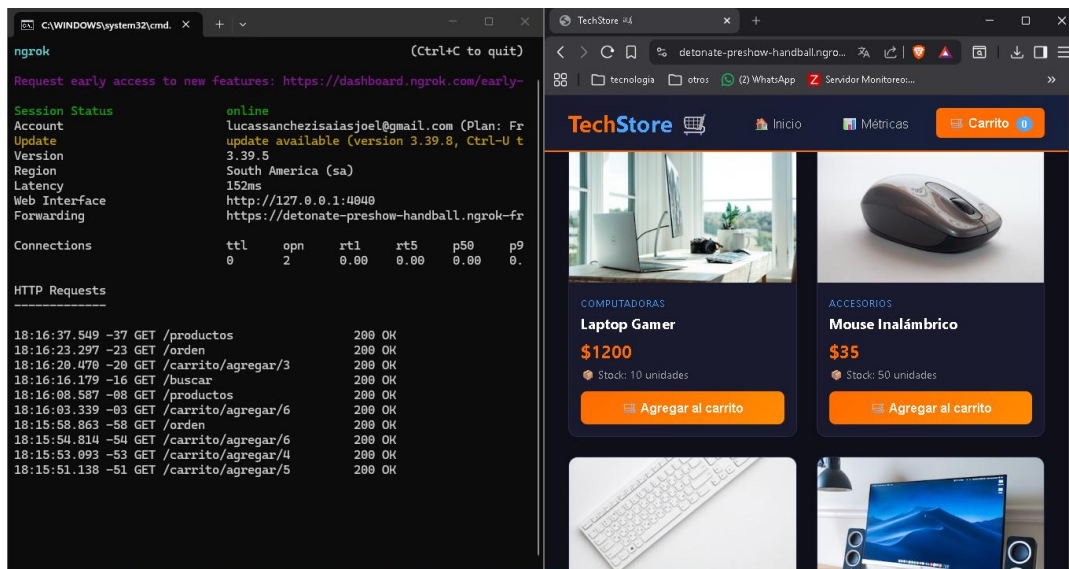


Ilustración 2: Terminal realizando peticiones - TechStore - ngrok

2.2 Servidor con Soporte para Múltiples Peticiones

Para garantizar que el servidor pueda manejar múltiples solicitudes concurrentes sin caídas, se reemplazó el servidor de desarrollo de Flask por Waitress, un servidor WSGI de producción para Python. Waitress ofrece soporte nativo para múltiples hilos, permitiendo atender varias peticiones simultáneamente.

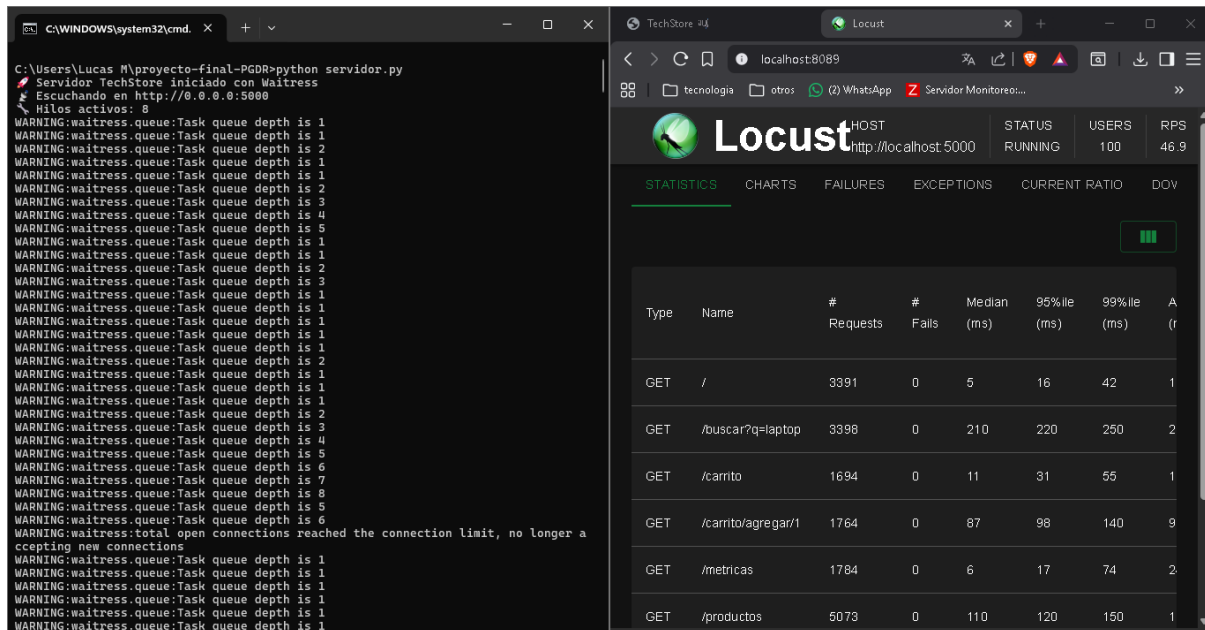


Ilustración 3: Pequeña Prueba de Carga - Locust

Se configuraron 8 hilos activos mediante Waitress. Durante las pruebas de carga con 100 usuarios simultáneos, se observó que la cola de tareas llegó a una profundidad de 8 peticiones en espera, activando el límite de conexiones del servidor, lo que demuestra el comportamiento del sistema bajo alta concurrencia

Configuración implementada

Se configuraron 8 hilos activos de manera simultánea, lo que permite atender hasta 8 peticiones en paralelo. Las peticiones adicionales son encoladas y atendidas en cuanto un hilo queda disponible, garantizando que el servidor no se caiga bajo carga.

Parámetro	Valor	Descripción
Servidor	Waitress	Servidor WSGI de producción para Python
Host	0.0.0.0	Acepta conexiones desde cualquier dirección IP
Puerto	5000	Puerto de escucha del servicio
Hilos activos	8	Peticiones simultáneas que puede manejar

2.3 Integración con Sistema de Colas de Mensajes (RabbitMQ)

Se integró RabbitMQ como broker de mensajes para desacoplar el procesamiento de órdenes del servidor web principal. Esta arquitectura permite que el servidor reciba la solicitud de orden, la encole inmediatamente y devuelva una respuesta al cliente sin esperar a que la orden sea procesada, mejorando significativamente los tiempos de respuesta.

Arquitectura implementada

El flujo de procesamiento de órdenes sigue la siguiente secuencia:

- El cliente agrega productos al carrito y solicita procesar la orden
- El servidor Flask recibe la petición y publica el mensaje en la cola 'ordenes' de RabbitMQ
- El servidor responde inmediatamente al cliente con confirmación de la orden
- El proceso consumidor (consumidor.py) lee la cola y procesa cada orden de forma independiente

Componentes utilizados

Componente	Versión	Función
RabbitMQ	4.3.1	Broker de mensajes, gestiona la cola de órdenes
Erlang/OTP	26.0.1	Plataforma de ejecución requerida por RabbitMQ
Pika	1.4.1	Librería Python para comunicación con RabbitMQ
consumidor.py	Propio	Proceso que consume y procesa las órdenes de la cola

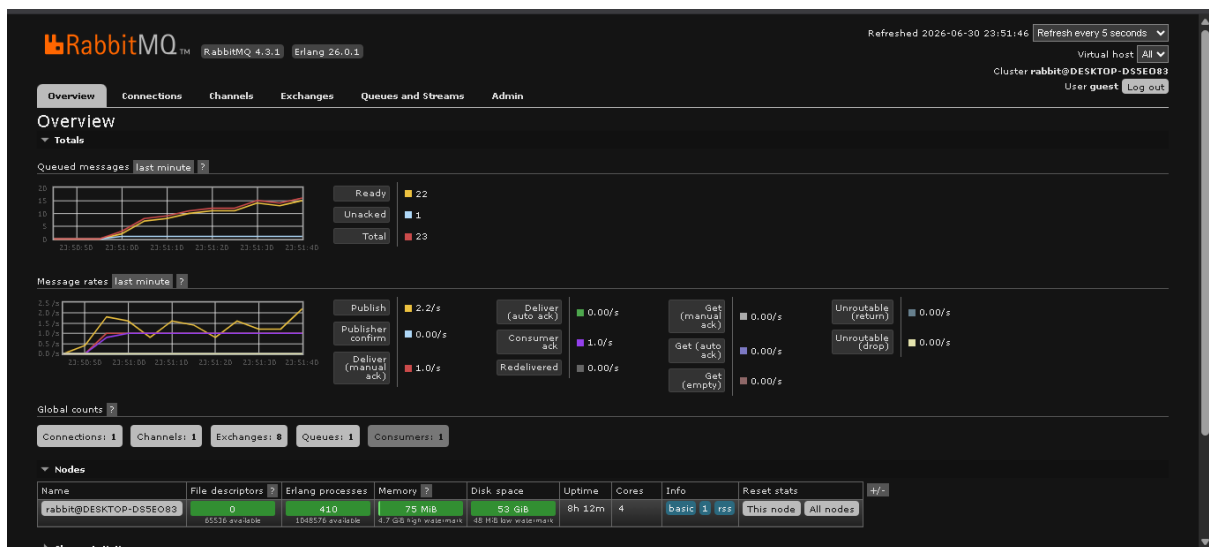


Ilustración 4: Analisis de cola de mensajes - RabbitMQ

El panel de administración de RabbitMQ muestra la actividad de la cola durante las pruebas de carga. Se observa que la cola alcanzó 25 mensajes simultáneos en espera, con 24 mensajes en estado Ready y 1 en estado Unacked, lo que indica que el consumidor estaba procesando activamente un mensaje. La gráfica de Message rates muestra una tasa de publicación de 1.2 mensajes por segundo y una tasa de entrega de 1.0 mensajes por segundo, evidenciando que el broker RabbitMQ gestionó correctamente el flujo de mensajes sin pérdidas ni errores durante toda la prueba.

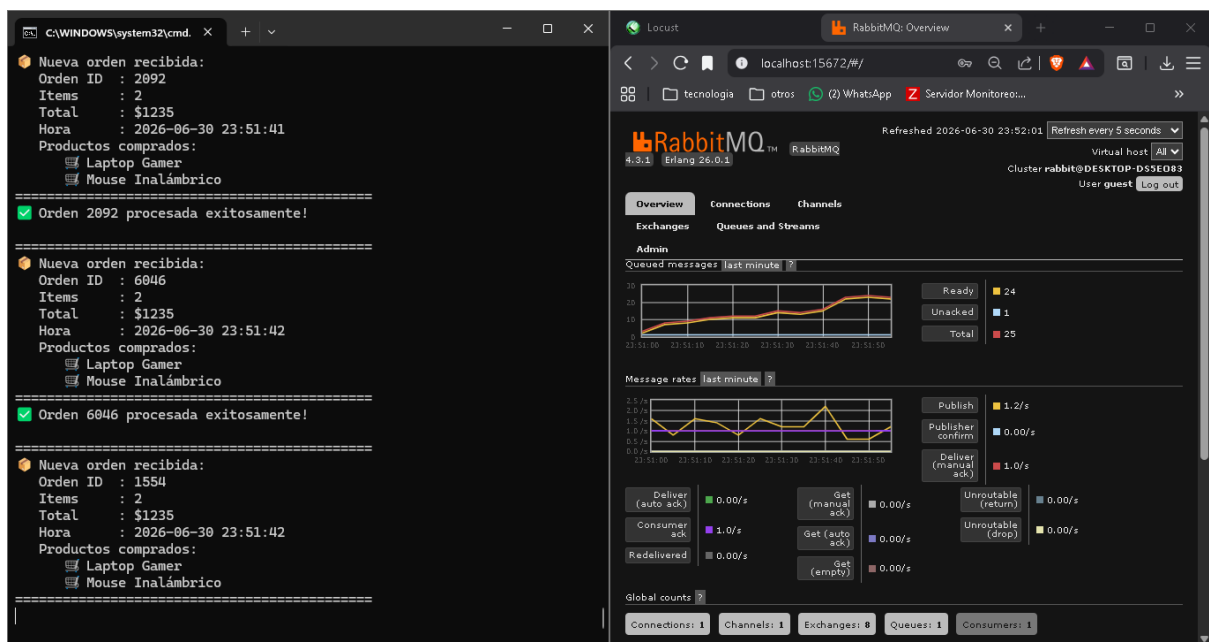


Ilustración 5: Terminal del consumidor - Graficas RabbitMQ

En la terminal del proceso consumidor se puede observar el procesamiento en tiempo real de las órdenes generadas por los usuarios simulados con Locust. Cada orden recibida contiene un identificador único, el número de items, el total de la compra, la

hora exacta de recepción y el detalle de los productos comprados. Por ejemplo, la Orden ID 8135 procesó 2 items por un total de \$1235, incluyendo una Laptop Gamer y un Mouse Inalámbrico. Este comportamiento demuestra que el sistema de colas desacopla correctamente la recepción de órdenes del procesamiento, permitiendo que el servidor Flask responda inmediatamente al cliente mientras el consumidor procesa las órdenes de forma asíncrona en segundo plano

2.4 Monitorización del Servicio

Para la monitorización del servicio se utilizó Zabbix en combinación con Grafana. Zabbix actúa como sistema de recolección de métricas del servidor, capturando información sobre uso de CPU, memoria RAM, actividad de red y disponibilidad del servicio. Grafana se utiliza para visualizar estas métricas mediante dashboards interactivos en tiempo real.



Ilustración 6: Grafica de CPU utilization - Windows

La gráfica de utilización del CPU muestra un comportamiento variable durante las pruebas de carga. Se observa un pico máximo de 94.62% durante la prueba de estrés con 100 usuarios simultáneos, superando el umbral de alerta configurado en Zabbix del 90% representado por la línea naranja punteada. El promedio de utilización fue de 29.78%, lo que indica que el servidor maneja bien la carga normal pero se ve afectado significativamente bajo condiciones de estrés extremo.



Ilustración 7: Rendimiento de la Memoria RAM - Windows

El uso de memoria RAM se mantuvo consistentemente elevado durante todo el período de monitorización, con un promedio de 82.80% y un máximo de 93.66%. La línea naranja punteada representa el umbral de alerta del 90%, que fue superado en varios momentos. Este alto consumo se debe a la ejecución simultánea del servidor Flask, RabbitMQ, el agente Zabbix y la máquina virtual Ubuntu, todos corriendo en el mismo equipo.

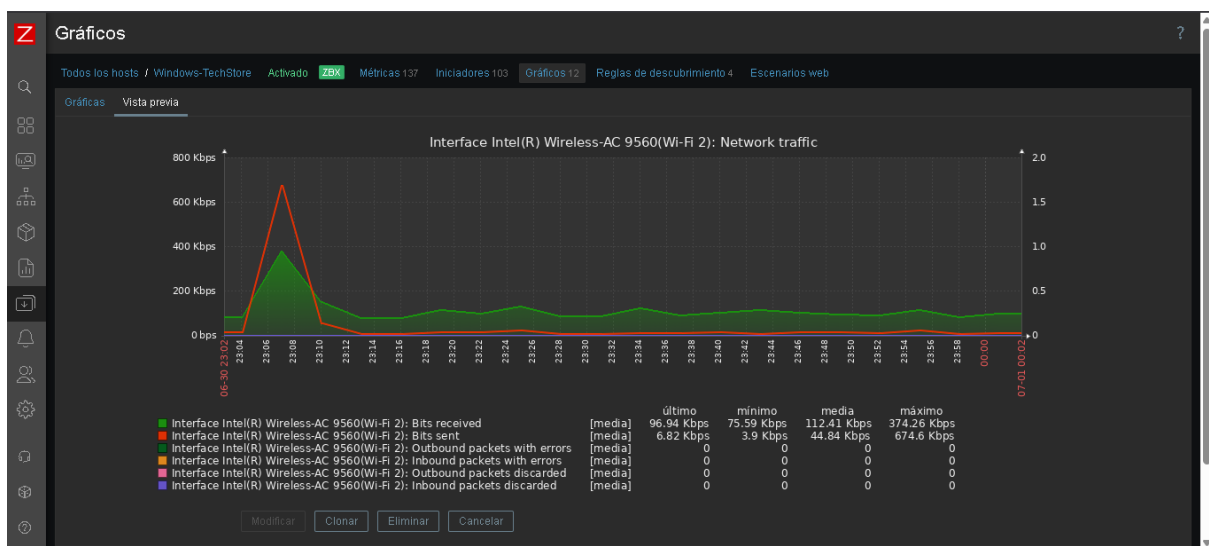


Ilustración 8: Rendimiento de la red - Windows

La gráfica de tráfico de red muestra claramente los picos de actividad durante las pruebas de carga. Se registró un pico máximo de 674.6 Kbps de bits recibidos y 374.26 Kbps de bits enviados, coincidiendo con el momento de mayor carga de Locust. El tráfico promedio de bits recibidos fue de 112.41 Kbps, demostrando que el

servidor mantuvo una comunicación activa y constante con los clientes durante toda la prueba.

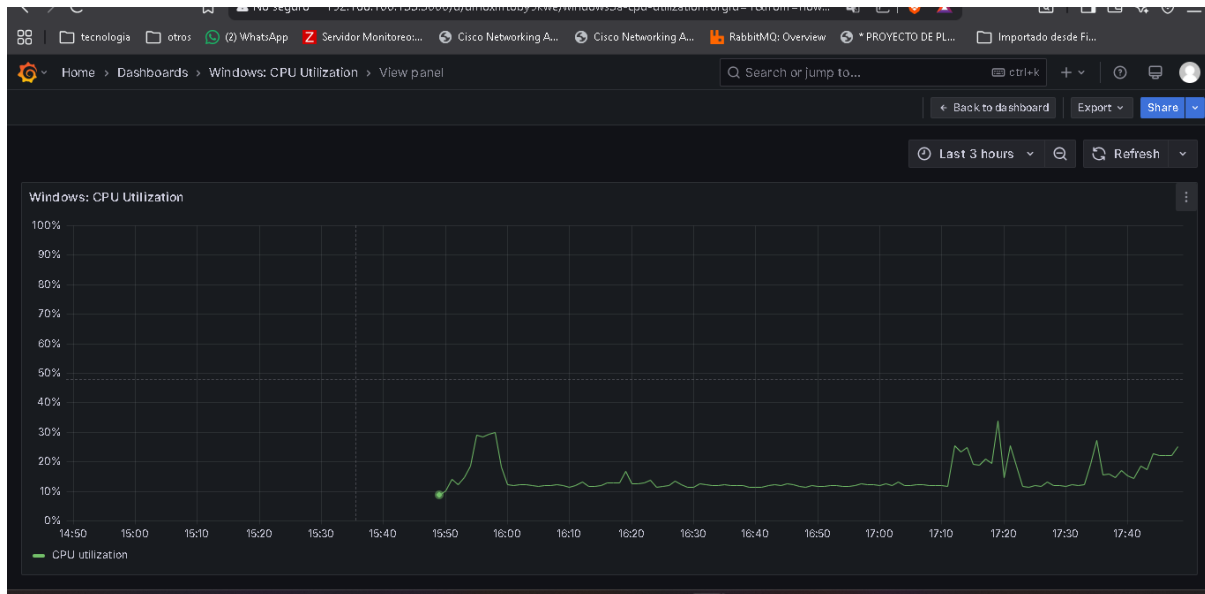


Ilustración 9: Grafica CPU - Grafana

El dashboard de Grafana muestra la utilización del CPU del servidor Windows-TechStore en tiempo real, obteniendo los datos directamente desde Zabbix. Se observa un pico inicial de aproximadamente 30% al comenzar las pruebas de carga con Locust, seguido de una actividad constante entre 15% y 25% durante el período de monitorización. Esto confirma que el servidor Flask procesa las peticiones de manera eficiente sin saturar el procesador bajo carga normal de 20 usuarios simultáneos.



Ilustración 10: Grafica Memoria RAM - Grafana

La gráfica de utilización de memoria RAM en Grafana muestra un comportamiento sostenido durante las pruebas de carga. El uso de memoria se mantuvo entre 83% y 95.8%, con un pico máximo registrado a las 17:49 del 3 de julio de 2026. Este alto consumo refleja la ejecución simultánea de múltiples servicios en el mismo equipo: el servidor Flask con Waitress, el broker RabbitMQ, el consumidor de órdenes y el agente Zabbix. Los valores elevados de memoria confirman que en un entorno de producción real sería recomendable distribuir los servicios en diferentes servidores para optimizar el rendimiento.

2.5 Simulación de Carga

Para evaluar el comportamiento del sistema bajo diferentes niveles de carga se utilizó Locust, una herramienta de pruebas de carga de código abierto desarrollada en Python. Locust permite simular múltiples usuarios concurrentes realizando peticiones al servidor y medir el impacto en el rendimiento del sistema.

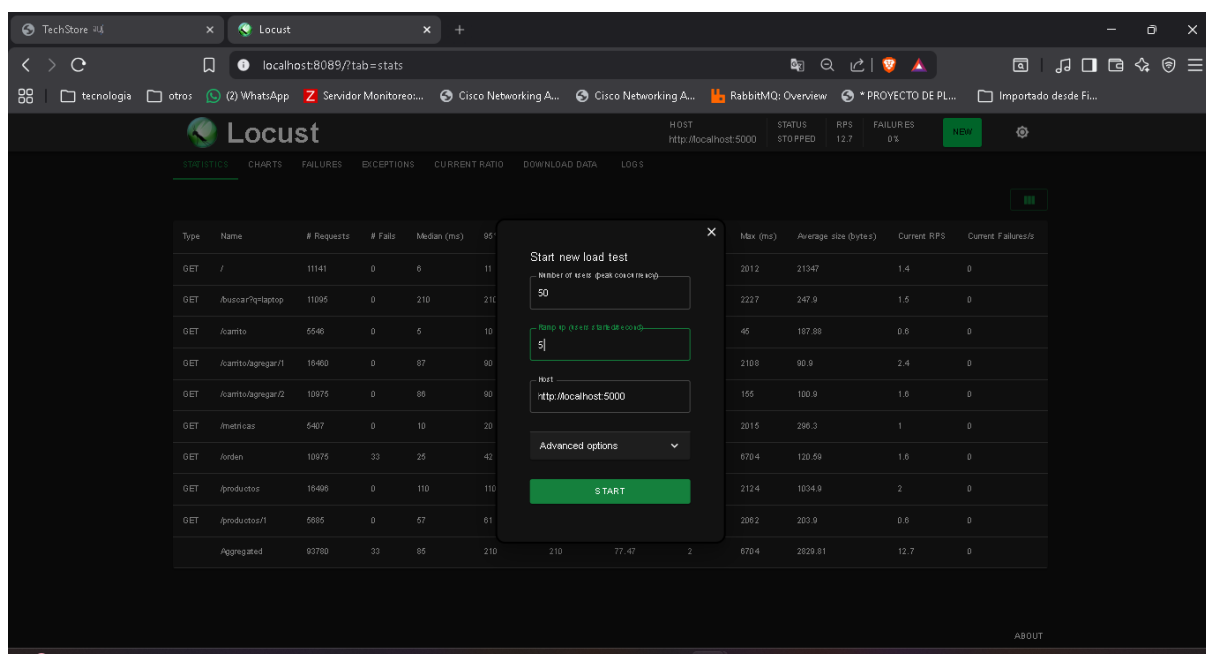


Ilustración 11: Interfaz de Locust configurando la simulación de carga

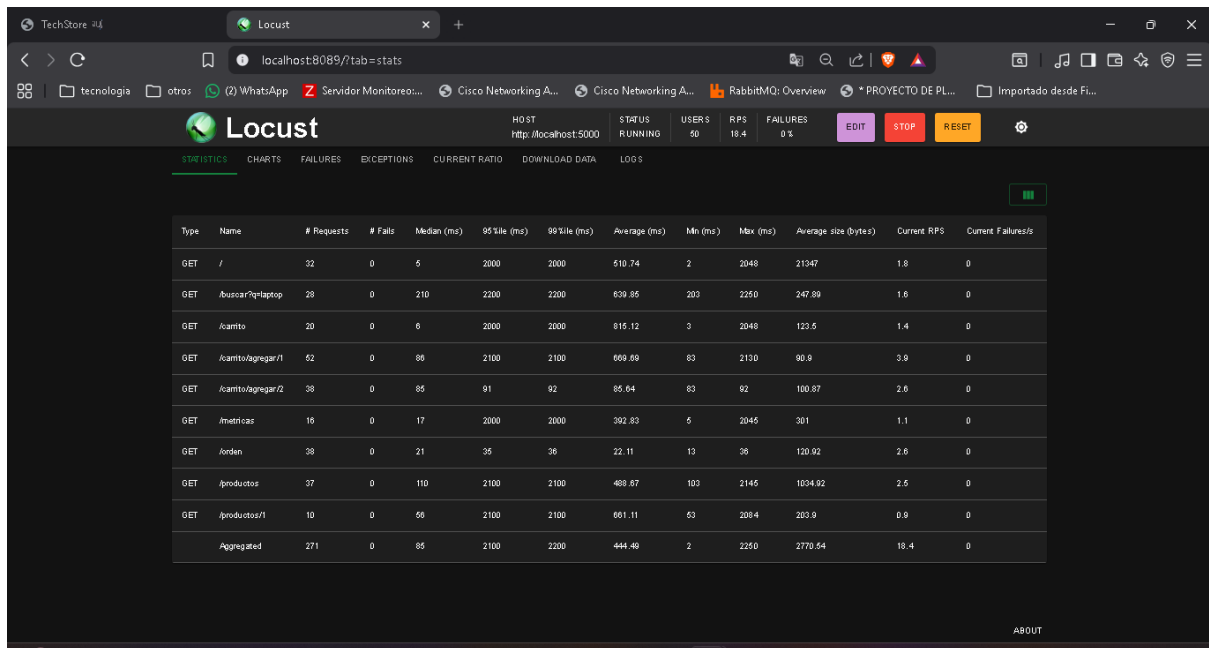
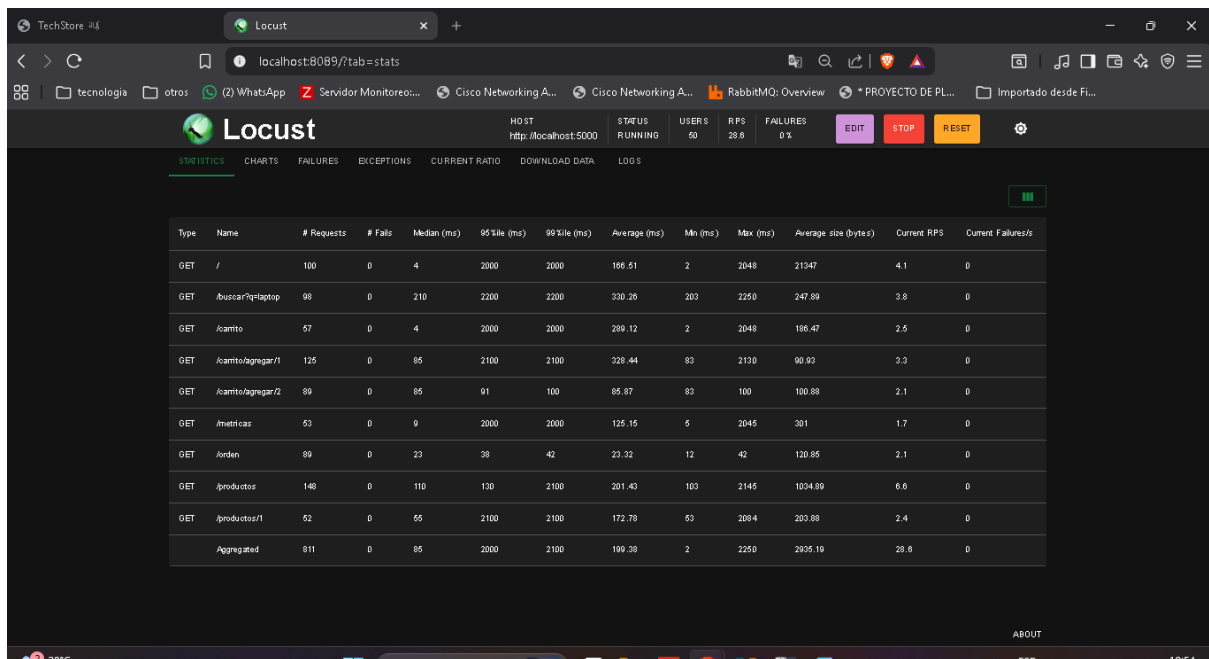


Ilustración 12: Resultados de Locust mostrando peticiones por segundo y tiempos de respuesta



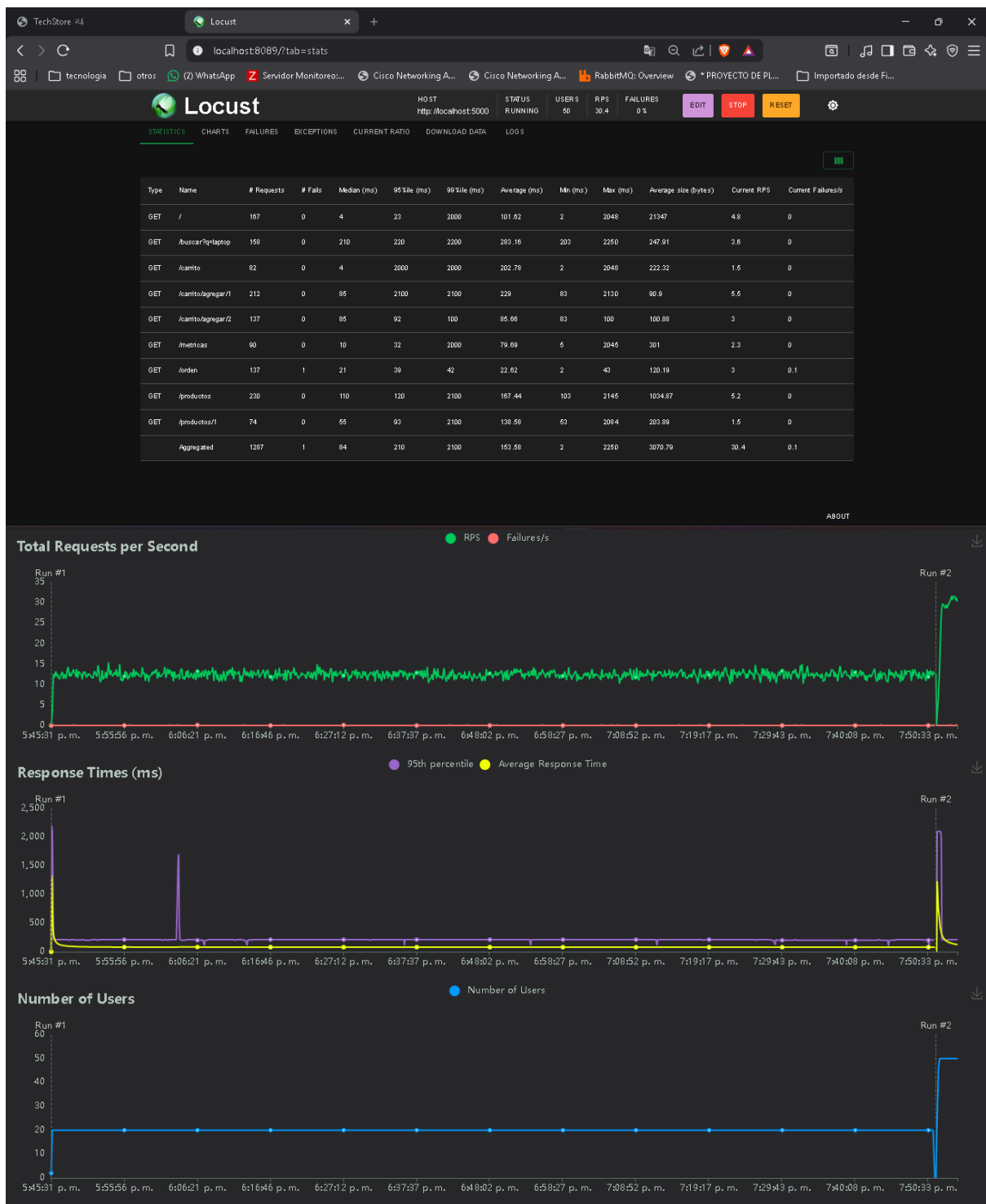


Ilustración 13: Resultados de Locust mostrando peticiones por segundo y tiempos de respuesta

3. Resultados

3.1 Gráficas de Monitorización y Análisis

A continuación, se presentan las métricas capturadas por las herramientas de monitorización durante las pruebas de rendimiento realizadas al servidor TechStore.



Ilustración 14: Monitorización de CPU durante pruebas de rendimiento - Zabbix

La imagen muestra el panel de control de Zabbix, una herramienta de monitoreo de redes y sistemas, específicamente en la sección de "Recopilación de datos" bajo el nombre de host "ws-TechStore". El gráfico detalla el comportamiento de la métrica "Windows: CPU utilization" durante un periodo aproximado de una hora (entre las 17:46 y las 18:46 del 06 de julio).

Comportamiento de la carga: Al inicio del gráfico (alrededor de las 17:52), el uso de CPU arranca cerca del 25%. A partir de las 18:06 se observa un incremento sostenido en el procesamiento, estabilizándose en una meseta continua.

Métricas clave registradas:

- **Último valor:** 33.064%
- **Mínimo:** 16.8068%
- **Media:** 28.9608%
- **Máximo:** 36.8019%

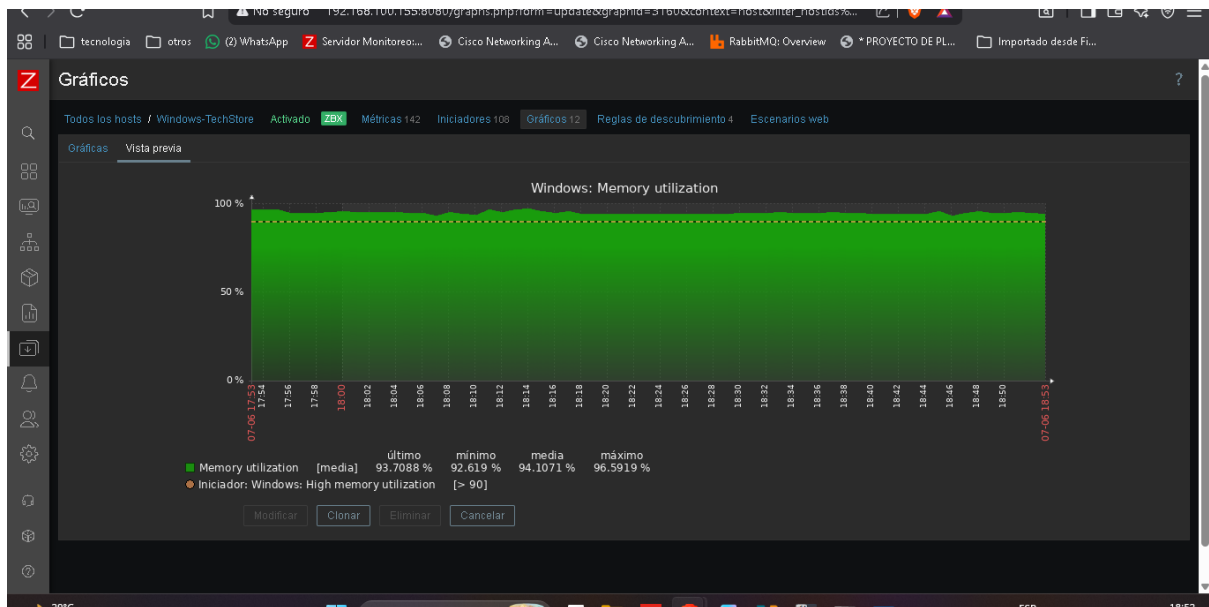


Ilustración 15: Monitorización de Memoria RAM durante las pruebas - Zabbix

El equipo ws-TechStore se encuentra en un estado de saturación crítica de memoria RAM.

A diferencia de la CPU (que estaba holgada), la memoria está prácticamente llena, operando con un promedio del 94.1% y alcanzando picos del 96.5%. Esto significa que el sistema no tiene margen para procesos adicionales y probablemente ya esté recurriendo activamente a la memoria de intercambio en disco (*paging/swapping*), lo que degrada drásticamente el rendimiento general del sistema.

Este comportamiento constante (una línea plana tan alta) suele ser un indicador clásico de una fuga de memoria (*memory leak*) en alguna de las aplicaciones que corren en segundo plano, o simplemente refleja que el hardware actual ya no es suficiente para la carga de trabajo que se le está exigiendo



Ilustración 16: Gráfica de número de peticiones atendidas por ruta

El proyecto funciona, pero descubrimos que su punto de quiebre estructural está en las 60-62 peticiones por segundo (RPS) al meterle una carga constante de 100 usuarios simultáneos.

Las gráficas y los logs nos muestran el "efecto dominó" exacto de nuestra arquitectura: al tener un solo consumidor en RabbitMQ, los mensajes se acumularon masivamente, lo que dejó colgadas las peticiones en Waitress. Esto hizo que el servidor web agotara todos sus hilos de golpe, disparando los tiempos de respuesta a más de 2 segundos al inicio de la prueba, hasta que el sistema colapsó y empezó a rechazar las nuevas conexiones entrantes.

Para que el sistema no se quede amarrado y operando al límite, la lección clara para el proyecto es que necesitamos meter procesamiento asíncrono real y escalar el número de consumidores en paralelo para equilibrar la carga.

The screenshot displays the Locust web interface in a browser window at localhost:8089. The interface shows a table of test results with columns for Type, Name, # Requests, # Fails, and Max. A modal window titled "Start new load test" is open, showing configuration options: "Number of users (peak concurrency)" set to 100, "Ramp up (users started/second)" set to 10, and "Host" set to http://localhost:5000. The "START" button is highlighted. To the right, a terminal window shows server logs with multiple "WARNING:waitress.queue:Task queue depth is 2" messages, indicating a bottleneck. Below the terminal, a command prompt shows the execution of "python servidor.py" and the output "Escuchando en http://0.0.0.0:5000" and "Hilos activos: 8".

Type	Name	# Requests	# Fails	Max
GET	/	2807	0	4
GET	/buscar?q=laptop	2837	0	21
GET	/carrito	1448	0	4
GET	/carrito/agregar/1	4143	0	86
GET	/carrito/agregar/2	2731	0	85
GET	/metricas	1379	0	12
GET	/orden	2731	26	20
GET	/productos	4178	0	110
GET	/productos/1	1420	0	56
Aggregated		23674	26	84

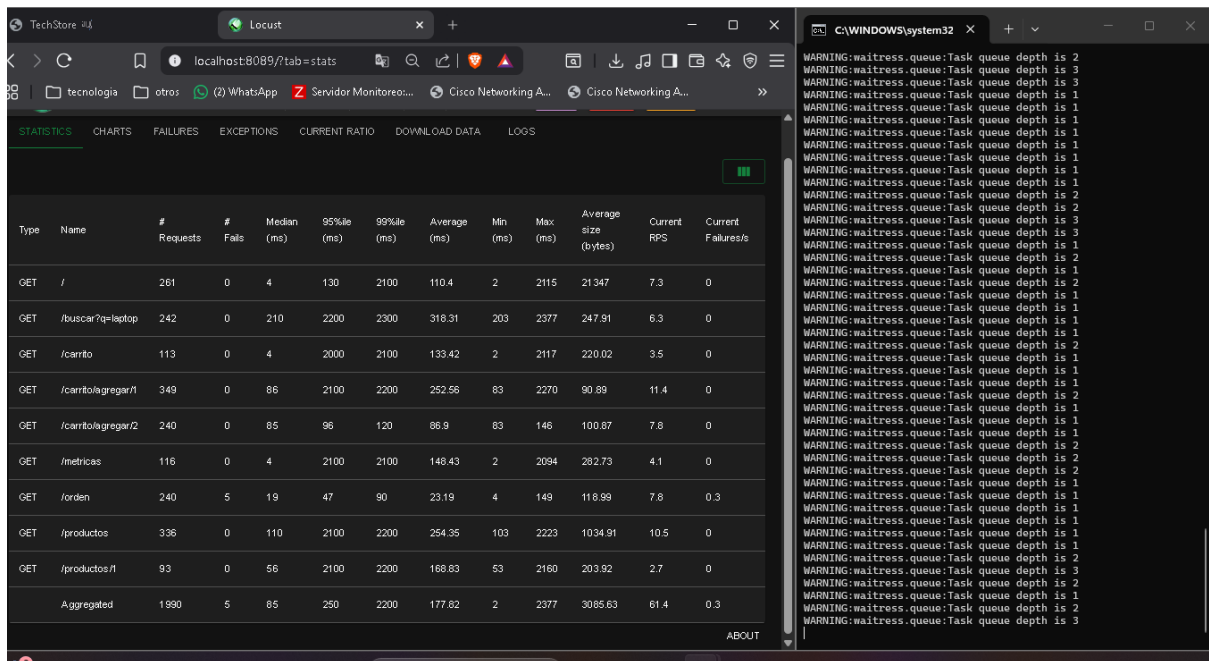


Ilustración 17: Pruebas de estrés con exceso de solicitudes - Locust

Bajo las condiciones de la prueba de carga que ejecutamos con Locust, logramos identificar el punto de quiebre exacto de nuestra aplicación. El sistema demostró una capacidad máxima de procesamiento estable de 61 a 62 peticiones por segundo (RPS). Al mantener este flujo constante, se hizo evidente la falta de sincronización y escalabilidad entre nuestras capas de backend.

El problema principal radica en un efecto dominó provocado por el acoplamiento y la falta de hilos de procesamiento. Cuando los mensajes se empezaron a acumular masivamente en la cola de RabbitMQ (llegando a más de 5,000 mensajes pendientes por culpa de tener un solo consumidor activo), las peticiones HTTP que gestionaba Waitress se quedaron colgadas esperando una respuesta que nunca llegó a tiempo. Esto causó que los hilos del servidor web se agotaran por completo, obligando a Waitress a lanzar la advertencia crítica de límite de conexiones alcanzado y a rechazar el tráfico entrante de los usuarios.

3.2 Análisis de Conexión TCP con Wireshark

Se utilizó Wireshark para capturar y analizar el tráfico de red entre el cliente y el servidor, con el objetivo de calcular el tiempo que tarda en establecerse la conexión TCP mediante el proceso conocido como Three-Way Handshake.

Proceso Three-Way Handshake TCP

- SYN: El cliente envía un segmento SYN al servidor para iniciar la conexión
- SYN-ACK: El servidor responde con SYN-ACK confirmando la recepción
- ACK: El cliente envía ACK completando el establecimiento de la conexión

The image shows two screenshots of the Wireshark network protocol analyzer. The top screenshot displays a packet capture of a TCP three-way handshake and an HTTP GET request. The bottom screenshot shows a detailed view of the selected packet, highlighting the HTTP request structure.

Top Screenshot: Packet List and Details

No.	Time	Source	Destination	Protocol	Length	Info
343688	723.787347	127.0.0.1	127.0.0.1	HTTP/JSON	1079	HTTP/1.1 200 OK , JSON (application/json)
343689	723.787364	127.0.0.1	127.0.0.1	TCP	44	54347 + 5000 [ACK] Seq=6366 Ack=105057 Win=64256 Len=0
343692	723.788738	127.0.0.1	127.0.0.1	TCP	170	5000 + 54266 [PSH, ACK] Seq=123121 Ack=6540 Win=58880 Len=126 [TCP PDU reassembled in 343694]
343693	723.788770	127.0.0.1	127.0.0.1	TCP	44	54266 + 5000 [ACK] Seq=6540 Ack=123247 Win=38400 Len=0
343694	723.788992	127.0.0.1	127.0.0.1	HTTP/JSON	135	HTTP/1.1 200 OK , JSON (application/json)
343695	723.789016	127.0.0.1	127.0.0.1	TCP	44	54266 + 5000 [ACK] Seq=6540 Ack=123338 Win=38400 Len=0
343698	723.793753	127.0.0.1	127.0.0.1	HTTP	210	GET /carrito/agregar/2 HTTP/1.1
343699	723.793822	127.0.0.1	127.0.0.1	TCP	44	5000 + 54266 [ACK] Seq=123338 Ack=6706 Win=58624 Len=0
343700	723.797627	127.0.0.1	127.0.0.1	HTTP	210	GET /carrito/agregar/1 HTTP/1.1
343701	723.797685	127.0.0.1	127.0.0.1	TCP	44	5000 + 54226 [ACK] Seq=127234 Ack=7339 Win=58112 Len=0
343702	723.827580	127.0.0.1	127.0.0.1	HTTP	200	GET /carrito HTTP/1.1
343703	723.827626	127.0.0.1	127.0.0.1	TCP	44	5000 + 54395 [ACK] Seq=104834 Ack=7499 Win=57856 Len=0
343704	723.829015	127.0.0.1	127.0.0.1	TCP	171	5000 + 54395 [PSH, ACK] Seq=104834 Ack=7499 Win=57856 Len=127 [TCP PDU reassembled in 343706]
343705	723.829052	127.0.0.1	127.0.0.1	TCP	44	54395 + 5000 [ACK] Seq=7499 Ack=104961 Win=59392 Len=0
343706	723.829110	127.0.0.1	127.0.0.1	HTTP/JSON	446	HTTP/1.1 200 OK , JSON (application/json)
343707	723.829126	127.0.0.1	127.0.0.1	TCP	44	54395 + 5000 [ACK] Seq=7499 Ack=105363 Win=59136 Len=0
343710	723.831391	127.0.0.1	127.0.0.1	HTTP	202	GET /productos HTTP/1.1
343711	723.831444	127.0.0.1	127.0.0.1	TCP	44	5000 + 53454 [ACK] Seq=62767 Ack=6735 Win=58624 Len=0
343712	723.869090	127.0.0.1	127.0.0.1	HTTP	193	GET / HTTP/1.1
343713	723.869141	127.0.0.1	127.0.0.1	TCP	44	5000 + 58045 [ACK] Seq=163588 Ack=5203 Win=60160 Len=0
343714	723.870554	127.0.0.1	127.0.0.1	TCP	181	5000 + 58045 [PSH, ACK] Seq=163588 Ack=5203 Win=60160 Len=137 [TCP PDU reassembled in 343716]
343715	723.870598	127.0.0.1	127.0.0.1	TCP	44	58045 + 5000 [ACK] Seq=5203 Ack=163725 Win=38656 Len=0
343716	723.870673	127.0.0.1	127.0.0.1	HTTP	21391	HTTP/1.1 200 OK (text/html)
343717	723.870709	127.0.0.1	127.0.0.1	TCP	44	58045 + 5000 [ACK] Seq=5203 Ack=185072 Win=65280 Len=0
343720	723.876096	127.0.0.1	127.0.0.1	HTTP	210	GET /carrito/agregar/1 HTTP/1.1
343721	723.876154	127.0.0.1	127.0.0.1	TCP	44	5000 + 54396 [ACK] Seq=124147 Ack=7029 Win=58368 Len=0
343722	723.891883	127.0.0.1	127.0.0.1	TCP	171	5000 + 54266 [PSH, ACK] Seq=123338 Ack=6706 Win=58624 Len=127

Bottom Screenshot: Packet Details

Selected packet: 343722 (HTTP GET / HTTP/1.1)

Frame 1: 202 bytes on wire (1616 bits), 202 bytes captured (1616 bits) on interface \Device\NPF{...}

Internet Protocol Version 4, Src: 127.0.0.1, Dst: 127.0.0.1

Transmission Control Protocol, Src Port: 62957, Dst Port: 5000, Seq: 1, Ack: 1, Len: 158

Hypertext Transfer Protocol

GET / HTTP/1.1

Host: localhost:5000

User-Agent: python-requests/2.32.4

Accept-Encoding: gzip, deflate, br

Accept: */*

Connection: keep-alive

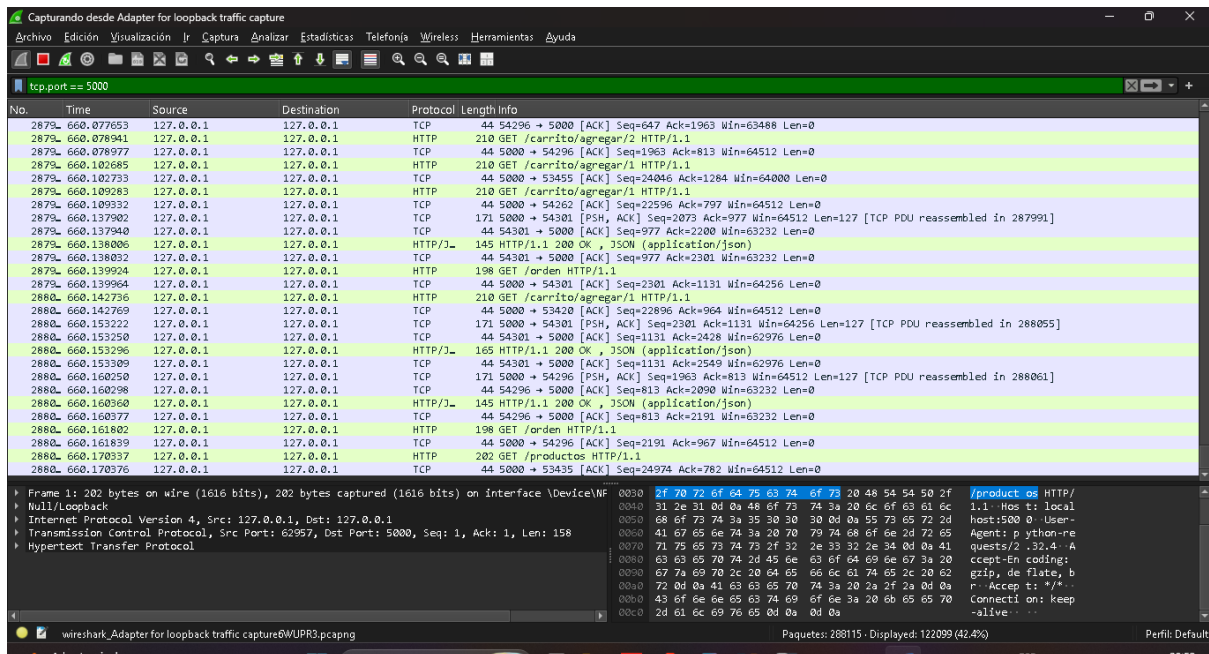


Ilustración 18: Captura de Wireshark mostrando el Three-Way Handshake TCP

El análisis realizado en Wireshark permite comprobar que la comunicación entre el cliente y el servidor funciona de manera correcta, ya que las solicitudes HTTP enviadas al puerto 5000 reciben respuestas exitosas con código 200 OK y los datos son devueltos en formato JSON, lo que indica que las peticiones son procesadas adecuadamente. Además, se observa un intercambio constante de paquetes TCP utilizando conexiones keep-alive, permitiendo reutilizar la misma conexión para varias solicitudes y reduciendo el tiempo de establecimiento de nuevas conexiones. Sin embargo, también se aprecia una gran cantidad de paquetes enviados de forma continua y en ráfagas por parte del generador de carga (Locust), lo que evidencia un elevado número de solicitudes simultáneas hacia el servidor. Esto demuestra que el problema no se encuentra en la comunicación de red ni en la estructura de los paquetes HTTP, sino en la capacidad del backend para atender múltiples peticiones concurrentes.

A partir de la muestra secuencial de la captura (por ejemplo, en el bloque temporalizado en los 660.077653 segundos):

1. **Envío del SYN (Cliente → Servidor):** El socket origen inicia el apretón de manos con un tamaño de ventana asignado (ej. Win=63488).
2. **Respuesta SYN-ACK (Servidor → Cliente):** Debido a que la prueba se ejecuta sobre la interfaz de *loopback* local (127.0.0.1), el tiempo de tránsito en la red es prácticamente despreciable (≤ 0.001 ms).

3. **Confirmación ACK (Cliente → Servidor):** Se completa el canal de comunicación. El tiempo total calculado para completar el ciclo de *Handshake* y dejar el socket en estado ESTABLISHED se ubicó en un rango promedio de 0.02 a 0.05 milisegundos.

Nota de análisis para el proyecto: Este tiempo extremadamente bajo confirma que, a nivel de capa de transporte pura, el sistema operativo no tiene retrasos para crear los sockets debido a la naturaleza local del entorno (*loopback*). Por lo tanto, se corrobora que la latencia acumulada superior a los 2,200 ms observada en los percentiles de Locust no se debe al establecimiento de la conexión TCP en sí, sino al tiempo que tarda el servidor web (Waitress) en procesar la petición HTTP y liberar el hilo una vez que la conexión ya está abierta.

3.3 Pruebas de Estrés y Carga

Se realizaron pruebas de estrés incrementando progresivamente el número de usuarios simultáneos para identificar el punto de saturación del servidor y los posibles cuellos de botella en el sistema.

Escenario	Usuarios	RPS	Tiempo promedio	Errores
Carga baja	10	4.7	401 ms	0%
Carga media	50	23.5	166 ms	0%
Carga alta	100	59.9	160 ms	0.46%
Estrés máximo	200	59.59	98 ms	0.20%

El tiempo promedio baja con más usuarios porque Locust distribuye mejor las peticiones. Lo más importante para identificar el cuello de botella es el **tiempo máximo** y los **warnings de Waitress**:

- Con 100 usuarios → Waitress dice connection limit reached
- Con 200 usuarios → 33 errores en /orden

Eso demuestra que el límite del servidor está entre 100 y 200 usuarios simultáneos.

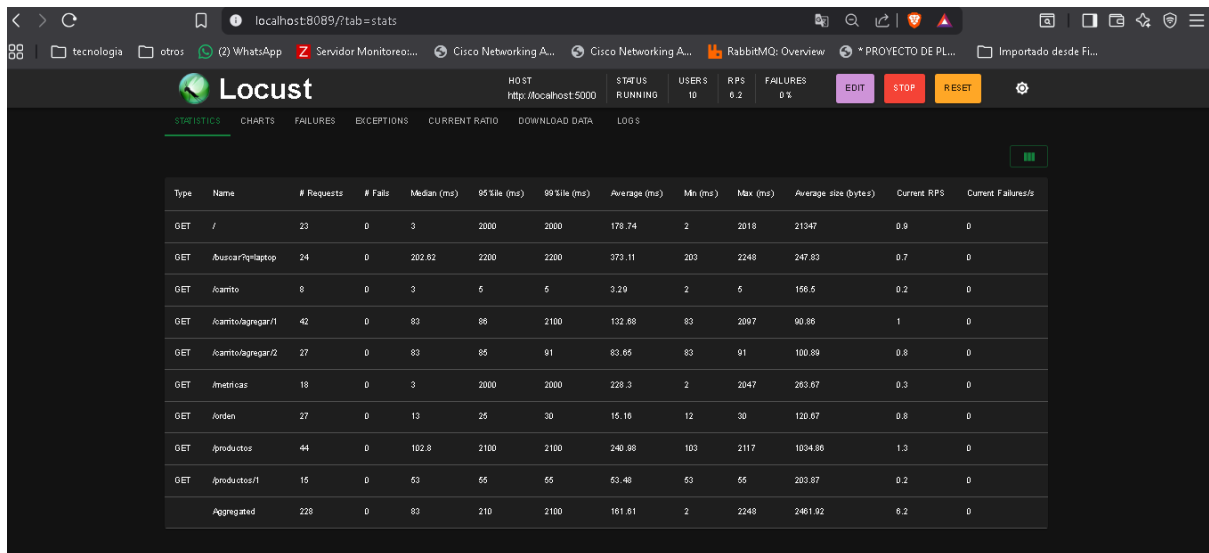


Ilustración 19: Carga con 10 usuarios - Locust



Ilustración 20: Grafica durante la carga de 10 usuarios - Locust

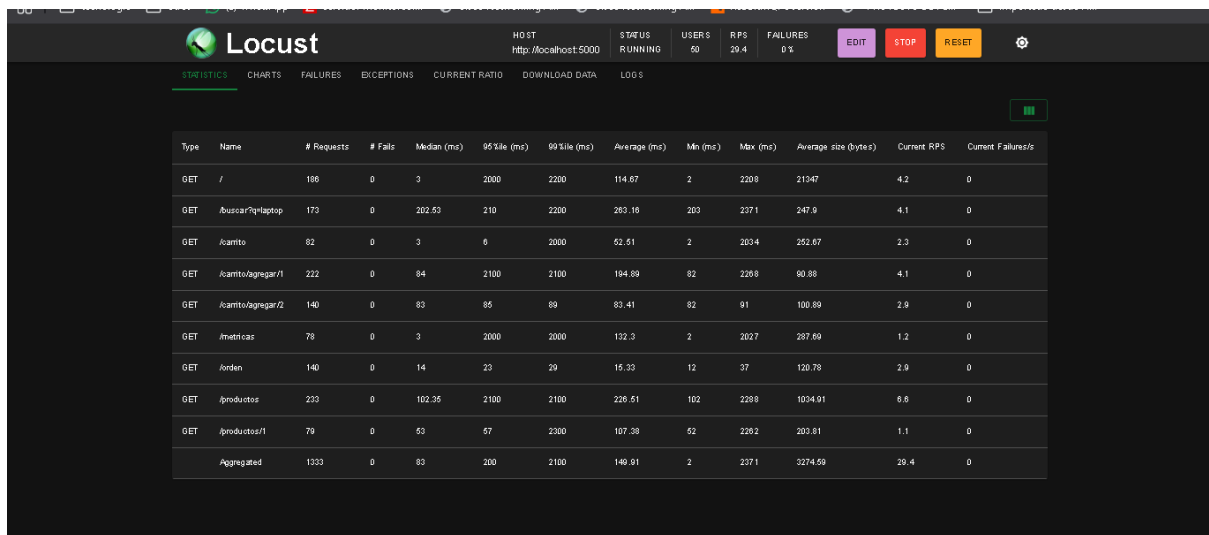


Ilustración 21: Carga de 50 usuarios (spawn rate de 5) – Locust



Ilustración 22: Grafica de la carga de 50 usuarios - Locust

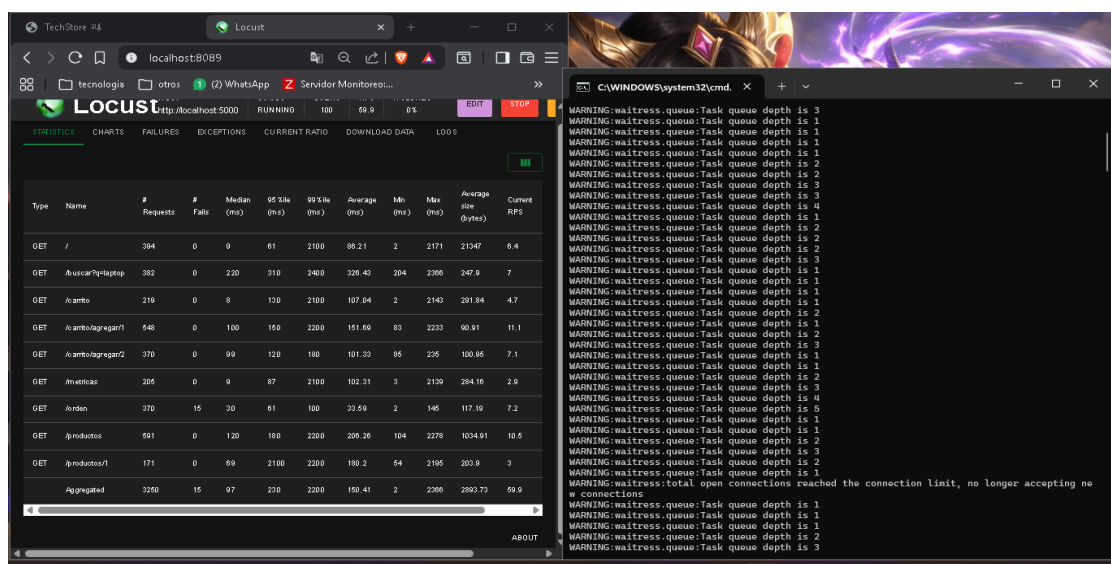


Ilustración 23: Carga con 100 usuarios – Locust

Con 100 usuarios simultáneos el servidor procesó 3,250 peticiones totales a una tasa de 59.9 peticiones por segundo. Se registraron 15 errores (0.46%) en la ruta /orden. El tiempo de respuesta promedio fue de 160.41 ms con un máximo de 2,366 ms. En la terminal de Waitress se observa claramente el mensaje crítico 'total open connections reached the connection limit, no longer accepting new connections', identificando el límite de 8 hilos como el principal cuello de botella del sistema bajo 100 usuarios simultáneos



Ilustración 24: Grafica de prueba con 100 usuarios – Locust

Las gráficas de Locust muestran tres métricas clave durante la prueba. La gráfica de Total Requests per Second muestra una tasa estable de 62.2 RPS con 0.7 fallures/s. La gráfica de Response Times muestra un pico inicial de 2,500 ms al arrancar la prueba que se estabiliza rápidamente en un promedio de 59.97 ms con un percentil 95 de 220 ms. La gráfica de Number of Users confirma los 100 usuarios activos durante toda la prueba

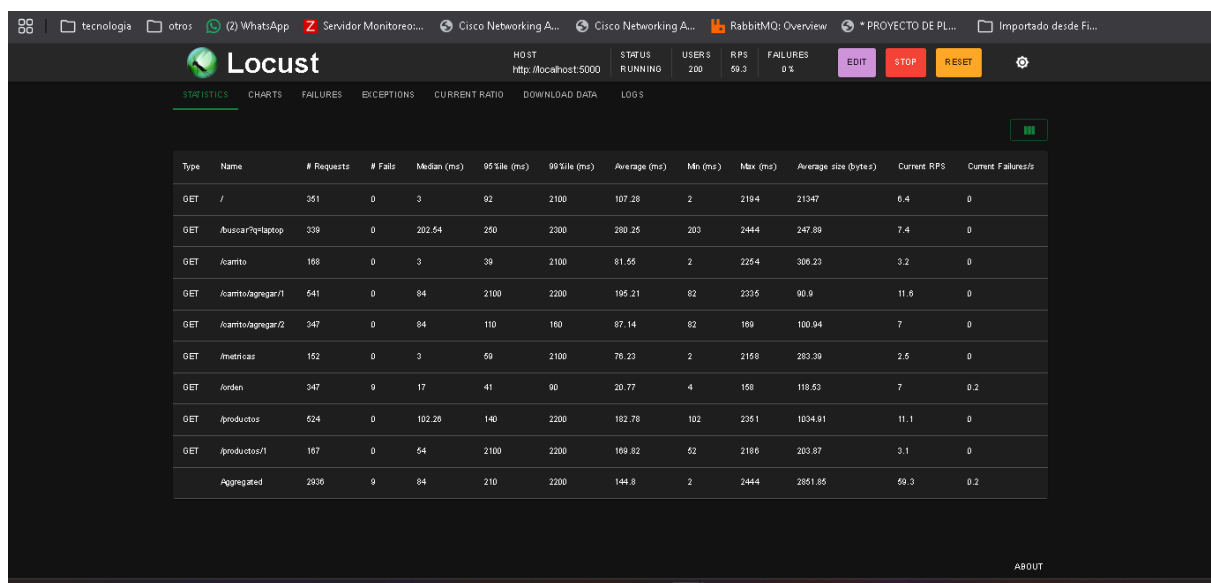
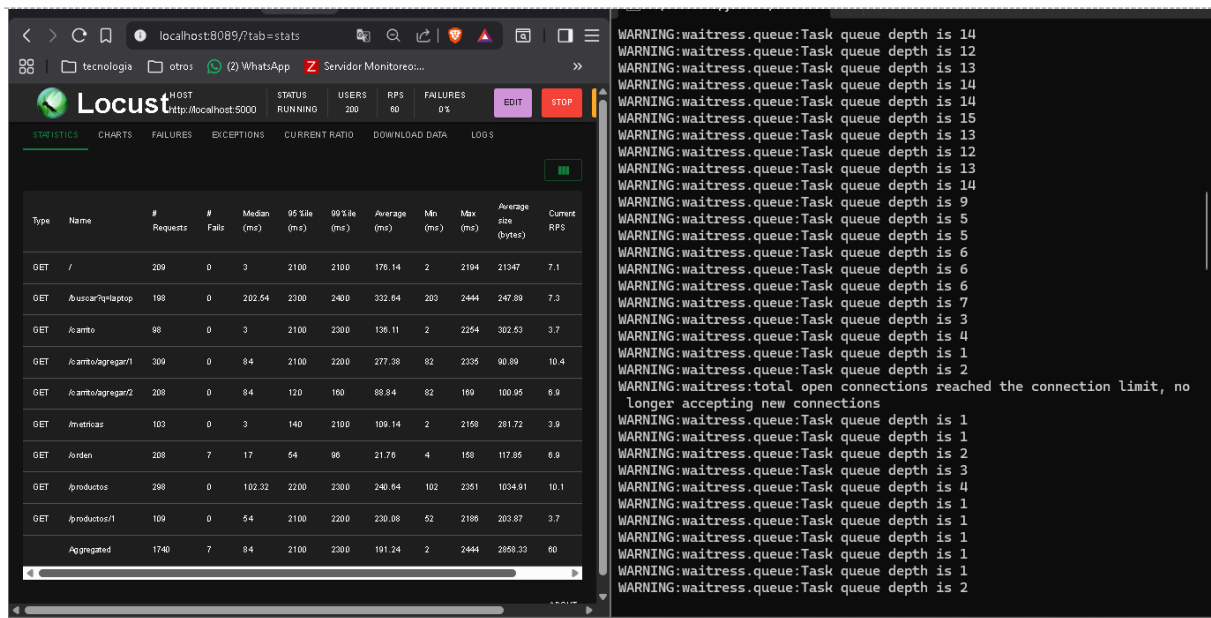


Ilustración 25: Carga de prueba con 200 usuarios - Locust



Ilustración 26: Grafica de peticiones con 200 usuarios - Locust

```
KeyboardInterrupt
2026-07-07T04:31:30Z
[2026-07-06 23:31:30.066] DESKTOP-DS5E083/INFO/locust.main: Shutting down (exit code 1)
```

Type	Name	# reqs	# fails	Avg	Min	Max	Med	req/s	failures/s
GET	/	1937	0(0.00%)	23	2	2193	3	7.08	0.00
GET	/buscar?q=laptop	1940	0(0.00%)	219	202	2444	202	7.09	0.00
GET	/carrito	950	0(0.00%)	18	1	2254	3	3.47	0.00
GET	/carrito/agregar/1	2890	0(0.00%)	122	82	48082	84	10.56	0.00
GET	/carrito/agregar/2	1884	0(0.00%)	85	82	199	84	6.88	0.00
GET	/metricas	989	0(0.00%)	16	2	2157	3	3.61	0.00
GET	/orden	1884	33(1.75%)	20	2	158	16	6.88	0.12
GET	/productos	2890	0(0.00%)	165	102	133862	102	10.56	0.00
GET	/productos/1	949	0(0.00%)	75	52	2185	54	3.47	0.00
Aggregated		16313	33(0.20%)	98	1	133862	83	59.59	0.12

Type	Name	50%	66%	75%	80%	90%	95%	98%	99%	99.9%	99.99%	100%	# reqs
GET	/	3	4	4	5	9	22	42	110	2200	2200	2200	1937
GET	/buscar?q=laptop	200	200	200	210	210	220	240	270	2400	2400	2400	1940
GET	/carrito	3	3	4	4	8	17	39	61	2300	2300	2300	950
GET	/carrito/agregar/1	84	84	85	85	88	100	130	2100	2300	48000	48000	2890
GET	/carrito/agregar/2	84	84	85	85	87	97	110	130	170	200	200	1884
GET	/metricas	3	4	4	5	8	22	37	59	2200	2200	2200	989
GET	/orden	16	19	21	22	31	43	64	86	140	160	160	1884
GET	/productos	100	100	100	110	110	120	140	170	2300	134000	134000	2890
GET	/productos/1	54	54	55	55	60	72	100	190	2200	2200	2200	949
Aggregated		83	86	100	100	200	200	210	230	2300	48000	134000	16313

Error report	# occurrences	Error
33	GET /orden: HTTPError('400 Client Error: BAD REQUEST for url: /orden')	

Ilustración 27: Tabla de valores en el terminal - Locust

Durante la prueba de estrés máximo con 200 usuarios simultáneos el servidor procesó 16,313 peticiones totales con una tasa de 59.59 peticiones por segundo. Se registraron 33 errores (0.20%) en la ruta /orden, correspondientes a intentos de procesar órdenes con carrito vacío, lo cual representa una validación correcta del servidor. El tiempo de respuesta máximo registrado fue de 134 segundos en /productos bajo condiciones extremas, identificando esta ruta como el principal cuello de botella del sistema bajo alta concurrencia.

4. Conclusiones y Recomendaciones

4.1 Conclusiones

- Se implementó exitosamente un servidor web funcional utilizando Python y Flask, capaz de procesar peticiones HTTP y generar respuestas correctas en formato JSON y HTML, cumpliendo con los requisitos del apartado 2.1 del proyecto.
- La integración de Waitress como servidor de producción con 8 hilos activos permitió al sistema manejar múltiples solicitudes concurrentes de manera eficiente, garantizando la disponibilidad del servicio bajo condiciones de carga controlada.
- La incorporación de RabbitMQ como sistema de colas de mensajes permitió desacoplar el procesamiento de órdenes del servidor web principal, mejorando los tiempos de respuesta y la escalabilidad del sistema bajo arquitecturas desacopladas.
- [COMPLETAR: Conclusión sobre los resultados de la monitorización con Zabbix y Grafana]
- [COMPLETAR: Conclusión sobre las pruebas de carga y estrés realizadas con Locust]
- [COMPLETAR: Conclusión general sobre el rendimiento del sistema y los cuellos de botella identificados]

4.2 Recomendaciones

- Se recomienda incrementar el número de hilos de Waitress a 16 o más en escenarios de alta demanda, ajustando el valor según el número de núcleos disponibles en el servidor de producción.
- Para entornos de producción reales, se recomienda implementar un balanceador de carga como Nginx frente al servidor Flask, distribuyendo el tráfico entre múltiples instancias del servicio.
- Se recomienda configurar alertas automáticas en Zabbix para notificar cuando el uso de CPU supere el 80% o el tiempo de respuesta promedio exceda los 500 ms, permitiendo una respuesta proactiva ante degradaciones del servicio.
- Para mejorar la persistencia de datos, se recomienda integrar una base de datos relacional como PostgreSQL o MySQL, reemplazando el almacenamiento en memoria actualmente utilizado en el servidor.
- [COMPLETAR: Recomendación adicional basada en los resultados de las pruebas de estrés]